

Mudcard

- **I struggled with the quiz three**
- **I am still a bit confused about how we get to 96.**
 - When you submit an answer for the quiz, you'll see an explanation of what went wrong and how to fix it.
- **I couldn't get the code running properly on my end (not in the sense that I put in the wrong code, it literally wouldn't run). How do I need to have everything downloaded formatting-wise and what tools do I need to use to run it?**
 - Check the PS1 announcement on Ed. There is a link to a google doc which describes how to set up the conda environment and run the code.
- **How to avoid and debug the string problems with "Doctorate" vs " Doctorate"?**
 - You have to check the data which often means you need to open the csv or excel file and take a look at it. :)
 - You also see the space when you print the header but it's a bit more difficult to notice.
 - The best thing you can do is to come up with tests before you start writing the code, the same approach we have in PS1.
- **Should we be preferentially learning pandas or polars? (Sorry, I may have missed when you spoke about it!) Or, what should it depend on?**
 - Yep, read through the lecture 2 notes again or watch the recording, it's in there and we discussed it.

Lecture 3

Working with data (step 0) continued

Learning objectives

By the end of the lecture, you will be able to

- filter columns
- merge and append data frames
- modify a dataframe

Pandas and Polars

- data are often distributed over multiple files/databases (e.g., csv and excel files, sql databases)
- each file/database is read into a pandas dataframe - last lecture
- you often need to filter dataframes by selecting specific rows - last lecture
- you often need to filter dataframes by selecting specific columns - today
- multiple dataframes need to be merged and appended - today
- dataframes sometimes need to be modified - today

Some notes and advice

- **ALWAYS READ THE HELP OF THE METHODS/FUNCTIONS YOU USE!**
- stackoverflow is your friend, use it! <https://stackoverflow.com/>
- you can also use generative AI to help you fix bugs
 - Gemini is supported by Brown OIT (see [here](#))
- [here](#) is an excellent review of the syntax differences between pandas and polars

Learning objectives

By the end of the lecture, you will be able to

- **filter columns**

- merge and append data frames
- modify a dataframe

```
In [ ]: import pandas as pd
df_pd = pd.read_csv('../data/adult_data.csv')

columns = df_pd.columns
#print(columns)

# select columns by column name
#print(df_pd[['age', 'hours-per-week']])
#print(columns[[1,5,7]])
#print(df_pd[columns[[1,5,7]]])

# select columns by index using iloc
#print(df_pd.iloc[:,3])

# select columns by index - not standard python indexing
#print(df_pd.iloc[:,[3,5,6]])

# select columns by index - standard python indexing
#print(df_pd.iloc[:, :2])
```

```
In [ ]: import polars as pl
df_pl = pl.read_csv('../data/adult_data.csv')

columns = df_pl.columns
print(columns)

# select columns by column name
#print(df_pl['age', 'hours-per-week'])
#print(df_pl.select(['age', 'hours-per-week'])) # use .select if you know the column names. it's often more
#print(columns[1:4]) # indices must be integers or slices
#print(df_pl[columns[1:4]])

# select columns by index, polars has no .iloc
#print(df_pl[:,3])

# select columns by index - not standard python indexing but it works
#print(df_pl[:, [3,5,6]])

# select columns by index - standard python indexing
#print(df_pl[:, :2])
```

Learning objectives

By the end of the lecture, you will be able to

- filter columns
- **merge and append data frames**
- modify a dataframe

How to merge dataframes in Pandas?

Merge - info on data points are distributed in multiple files

```
In [ ]: # We have two datasets from two hospitals

hospital1 = {'ID': ['ID1', 'ID2', 'ID3', 'ID4', 'ID5', 'ID6', 'ID7'], 'col1': [5,8,2,6,0,2,5], 'col2': ['y', 'j', 'w', 'l', 'p', 'e', 'l']}
df1 = pd.DataFrame(data=hospital1)
print(df1)

hospital2 = {'ID': ['ID2', 'ID5', 'ID6', 'ID10', 'ID11'], 'col3': [12,76,34,98,65], 'col2': ['q', 'u', 'e', 'l', 'p']}
df2 = pd.DataFrame(data=hospital2)
print(df2)

In [ ]: # we are interested in only patients from hospital1
df_left = df1.merge(df2, how='left', on='ID') # IDs from the left dataframe (df1) are kept
#print(df_left)
```

```
# we are interested in only patients from hospital2
#df_right = df1.merge(df2,how='right',on='ID') # IDs from the right dataframe (df2) are kept
#df_right = df2.merge(df1,how='left',on='ID')
#print(df_right)

# we are interested in patients who were in both hospitals
#df_inner = df1.merge(df2,how='inner',on='ID') # merging on IDs present in both dataframes
#print(df_inner)

# we are interested in all patients who visited at least one of the hospitals
df_outer = df1.merge(df2,how='outer',on='ID') # merging on IDs present in any dataframe
print(df_outer)
```

How to append dataframes in pandas?

Append - new data comes in over a period of time. E.g., one file per month/quarter/fiscal year etc.

You want to combine these files into one data frame.

```
In [ ]: #df_append = pd.concat([df1,df2]) # note that rows with ID2, ID5, and ID6 are duplicated! Indices are duplicated
#print(df_append)

#df_append = pd.concat([df1,df2],ignore_index=True) # note that rows with ID2, ID5, and ID6 are duplicated
#print(df_append)

d3 = {'ID':['ID23','ID94','ID56','ID17'],'col1':['rt','h','st','ne'],'col2':[23,86,23,78]}
df3 = pd.DataFrame(data=d3)
#print(df3)

df_append = pd.concat([df1,df2,df3],ignore_index=True) # multiple dataframes can be appended
print(df_append)
```

How to merge/join dataframes in Polars?

```
In [ ]: hospital1 = {'ID':['ID1','ID2','ID3','ID4','ID5','ID6','ID7'],'col1':[5,8,2,6,0,2,5],'col2':['y','j','w'],'col3':[1,2,3,4,5,6,7]}
df1 = pl.DataFrame(data=hospital1)
#print(df1)

hospital2 = {'ID':['ID2','ID5','ID6','ID10','ID11'],'col3':[12,76,34,98,65],'col2':['q','u','e','l','p'],'col1':[1,2,3,4,5]}
df2 = pl.DataFrame(data=hospital2)
#print(df2)

# left join
#df_left = df1.join(df2,how='left',on='ID') # IDs from the left dataframe (df1) are kept
#print(df_left)

# right join
#df_right = df1.join(df2,how='right',on='ID') # IDs from the right dataframe (df2) are kept
#df_right = df2.join(df1,how='left',on='ID')
#print(df_right)

# inner join
#df_inner = df1.join(df2,how='inner',on='ID') # merging on IDs present in both dataframes
#print(df_inner)

# outer join is called a full join
#df_outer = df1.join(df2,how='full',on='ID') # merging on IDs present in any dataframe
#print(df_outer)
```

Quiz

```
In [ ]: raw_data_1 = {
    'subject_id': ['1', '2', '3', '4', '5'],
    'first_name': ['Alex', 'Amy', 'Allen', 'Alice', 'Ayoung'],
    'last_name': ['Anderson', 'Ackerman', 'Ali', 'Aoni', 'Atiches']}

raw_data_2 = {
    'subject_id': ['6', '7', '8', '9', '10'],
    'first_name': ['Billy', 'Brian', 'Bran', 'Bryce', 'Betty'],
```

```

    'last_name': ['Bonder', 'Black', 'Balwner', 'Brice', 'Btisan']}

raw_data_3 = {
    'subject_id': ['1', '2', '3', '4', '5', '7', '8', '9', '10', '11'],
    'test_id': [51, 15, 15, 61, 16, 14, 15, 1, 61, 16]}

# Create three data frames from raw_data_1, 2, and 3.
# Append the first two data frames and assign it to df_append.
# Merge the third data frame with df_append such that only subject_ids from df_append are present.
# Assign the new data frame to df_merge.
# How many rows and columns do we have in df_merge?

```

Always check that the resulting dataframe is what you wanted to end up with!

- small toy datasets are ideal to test your code.

If you need to do a more complicated dataframe operations, check out `pd.concat()` and `pl.concat()`!

Learning objectives

By the end of the lecture, you will be able to

- filter columns
- merge and append data frames
- **modify a dataframe**

Do not EVER overwrite the original data files!

Always save the modified dataframe to a new file!

Why do we need to modify the dataset?

- feature engineering
 - generating new features (adding new informative columns) can improve the performance of the ML model
- fix dataset issues
 - there might be data quality issues that need to be manually fixed
 - typos, missing values, etc

```

In [ ]: import pandas as pd
df_pd = pd.read_csv('../data/adult_data.csv')
#print(df_pd.head())

# let's generate a new feature
# is immigrant? False (0) if the person's home country is the USA, True (1) otherwise
# such a feature has only two categories but it might be pretty informative for some purposes
df_pd['is_immigrant'] = df_pd['native-country'] != 'United-States'

print(df_pd.head())

```

Mud card

In []: